CS791 Probablistic Foundations of Artificial Intelligence

Assignment 1

Abhilasha Sharma Suman (23B1011)

Jigyasa Chouhan (23B1083)

S Ramachandran (23B1052)

Indian Institute of Technology, Bombay

Autumn 2025

Contents

1 Inference in FHMM **2**
- 1.1 Background . . . 2
- 1.2 Approach . . . 2
  - 1.2.1 Junction Tree Construction and Clique Formation . . . 2
  - 1.2.2 Time Complexity Analysis . . . 3
  - 1.2.3 Partition Function (get_z_value) . . . 3
  - 1.2.4 Marginal Probabilities (compute_marginals) . . . 4
  - 1.2.5 Top-K Assignments (compute_top_k) . . . 4
- 1.3 Implementation . . . 4
  - 1.3.1 get_z_value Implementation . . . 4
  - 1.3.2 compute_marginals Implementation . . . 6
  - 1.3.3 compute_top_k Implementation . . . 9

2 Loopy Belief Propagation **11**
- 2.1 Background . . . 11
- 2.2 Convergence & Complexity . . . 11
- 2.3 Implementation . . . 12

3 Turbo Codes **13**
- 3.1 Background . . . 13
- 3.2 Trellis Generation . . . 13
- 3.3 Viterbi Algorithm . . . 14
- 3.4 Turbo Codes and BCJL Algorithm . . . 14
- 3.5 Comparing Performance . . . 15

1 Inference in FHMM

1.1 Background

Message passing, also known as belief propagation, is a fundamental algorithm for performing efficient inference in graphical models. It operates by passing messages between nodes (or cliques in the case of junction trees) that contain local information about the variables they represent.

The key insight behind message passing is that global inference can be decomposed into local computations that are coordinated through the exchange of messages. There are two main variants:

- **Sum-Product Algorithm**: Used for computing marginal probabilities and partition functions. It involves summing over possible states of variables.
- **Max-Product Algorithm**: Used for finding the most probable configuration (MAP assignment). It involves taking maximum values instead of sums.

In the context of junction trees, message passing proceeds along the edges of the tree, with each message representing the marginalized information from one clique to its neighbor. The algorithm ensures that after all messages are passed, each clique contains the correct joint distribution over its variables conditioned on the evidence.

1.2 Approach

1.2.1 Junction Tree Construction and Clique Formation

The junction tree is constructed specifically for the Factorial HMM structure with the following clique types:

- **Observation Cliques (C_t)**: For each time step $t = 1$ to T, we create a clique containing all hidden variables at time t and the observation at time t:

$$C_t = \{H_t^1, H_t^2, \ldots, H_t^M, O_t\}$$

  These cliques capture the emission probabilities $P(O_t|H_t^1, \ldots, H_t^M)$.

- **Separation Cliques (S_t^n)**: Between each pair of consecutive observation cliques C_t and C_{t+1}, we create M separation cliques:

$$S_t^n = \{H_t^n, H_t^{n+1}, \ldots, H_t^M, H_{t+1}^1, H_{t+1}^2, \ldots, H_{t+1}^n\}$$

  for $n = 1$ to M. These cliques capture the transition probabilities $P(H_{t+1}^n|H_t^n)$ for each chain.

The junction tree has a **linear chain structure**:

$$C_1 \to S_1^1 \to S_1^2 \to \cdots \to S_1^M \to C_2 \to S_2^1 \to \cdots \to C_T$$

This construction ensures that:

1. Each clique has size $M + 1$ (manageable for small M)
2. The running intersection property is satisfied
3. All original factors are contained in some clique

1.2.2 Time Complexity Analysis

The time complexity of the inference algorithms is determined by:

- **Clique size**: Each clique contains $M + 1$ variables
- **State space**: Each variable has K possible states
- **Number of cliques**: $T + (T - 1) \times M = O(T \cdot M)$

The dominant operations and their complexities:

1. **Message computation between two cliques**:
$$O(K^{M+1})$$
Since each clique has $M + 1$ variables, and we need to iterate over all possible assignments.

2. **Total message passing**:
$$O(T \cdot M \cdot K^{M+1})$$
There are $O(T \cdot M)$ edges in the junction tree, and each message computation is $O(K^{M+1})$.

3. **Partition function computation**:
$$O(T \cdot M \cdot K^{M+1})$$
Same as message passing since it involves a forward pass.

4. **Marginal computation**:
$$O(T \cdot M \cdot K^{M+1})$$
Forward-backward passes have the same complexity as single passes.

5. **Top-k assignments**:
$$O(k \cdot T \cdot M \cdot K^{M+1})$$
Maintaining k-best paths increases complexity by factor k.

The complexity is **exponential in the number of factors** M but **linear in the time steps** T. This makes the approach feasible for FHMMs with small M but long sequences.

1.2.3 Partition Function (get_z_value)

The partition function Z represents the total probability of the observed evidence and is computed using the sum-product algorithm on the junction tree:

1. **Forward Pass**: Messages are passed from left to right through the junction tree chain: $C_1 \to S_1^1 \to S_1^2 \to \cdots \to S_1^M \to C_2 \to \cdots \to C_T$

2. **Message Computation**: For each edge between cliques X and Y, the message from X to Y is computed by:
$$\text{message}_{X \to Y}(\mathbf{s}) = \sum_{\mathbf{u}} \psi_X(\mathbf{s}, \mathbf{u})$$
where $\mathbf{s}$ represents the separation set variables (shared between X and Y), $\mathbf{u}$ represents variables unique to X, and ψ_X is the potential function of clique X.

3. **Message Incorporation**: Each incoming message is multiplied into the receiving clique's potential function.

4. **Final Computation**: After all forward messages are passed, Z is obtained by summing over all variables in the final clique C_T:
$$Z = \sum_{\mathbf{v}} \psi_{C_T}^{\text{final}}(\mathbf{v})$$
where $\psi_{C_T}^{\text{final}}$ is the potential of C_T after incorporating all incoming messages.

1.2.4 Marginal Probabilities (compute_marginals)

The marginal probabilities for each hidden variable are computed using forward-backward message passing:

1. **Forward Pass**: Same as for Z computation, storing all forward messages.
2. **Backward Pass**: Messages are passed from right to left through the junction tree, following the reverse direction: $C_T \to S_{T-1}^M \to \cdots \to S_{T-1}^1 \to C_{T-1} \to \cdots \to C_1$
3. **Belief Computation**: For each clique, the belief (posterior distribution) is computed by multiplying:
   - The clique's original potential
   - All incoming forward messages (from left)
   - All incoming backward messages (from right)
4. **Marginal Extraction**: For each hidden variable H_t^m, we:
   - Find a clique that contains H_t^m
   - Marginalize the clique's belief over all other variables
   - Normalize to obtain $P(H_t^m|\mathbf{o})$

The marginals are returned in the order: $H_1^1, H_1^2, \ldots, H_1^M, H_2^1, H_2^2, \ldots, H_T^M$.

1.2.5 Top-K Assignments (compute_top_k)

The top-K most probable assignments are found using the max-product algorithm with k-best path tracking:

1. **Max-Product Messages**: Instead of summing, we take maximum values:

$$\text{message}_{X \to Y}(\mathbf{s}) = \max_{\mathbf{u}} \psi_X(\mathbf{s}, \mathbf{u})$$

2. **K-Best Tracking**: For each message, we maintain the K best values and the assignments that produced them (backpointers).
3. **Backtracking**: Starting from the final clique, we work backwards through the junction tree, using the stored backpointers to reconstruct the K most probable complete assignments.
4. **Probability Computation**: For each reconstructed assignment $\mathbf{h}$, we compute its joint probability with the evidence:

$$P(\mathbf{h}, \mathbf{o}) = \prod_{i=1}^{N} \psi_{C_i}(\mathbf{v}_{C_i})$$

where N is the number of cliques and $\mathbf{v}_{C_i}$ are the variables in clique C_i.

1.3 Implementation

1.3.1 get_z_value Implementation

The partition function computation follows the sum-product algorithm along the junction tree:

```
def get_z_value(self):
    # Make a copy of cliques to avoid modifying the original potentials
    working_cliques = [clique.copy() for clique in self.cliques]

    # Forward pass: from left to right
    for i in range(len(working_cliques) - 1):
        from_clique = working_cliques[i]
        to_clique = working_cliques[i + 1]

        # Compute message from from_clique to to_clique
        message = self.compute_message(from_clique, to_clique)

        # Incorporate message into to_clique's potential
        sepset_vars = self.get_sepset_variables(from_clique, to_clique)
        old_potential_func = to_clique['potential_func']

        def new_potential_func(assignments, old_func=old_potential_func,
                               msg=message, sep_vars=sepset_vars):
            # Get sepset assignment
            sep_assignment = tuple(assignments[var] for var in sep_vars)
            # Get message value for this sepset assignment
            message_value = msg.get(sep_assignment, 0.0)
            # Multiply with original potential
            return old_func(assignments) * message_value

        to_clique['potential_func'] = new_potential_func

    # Sum over all variables in the last clique to get Z-value
    last_clique = working_cliques[-1]
    last_vars = last_clique['variables']

    # Create all possible assignments for the last clique
    assignments_list = []
    for var in last_vars:
        if var.startswith('H_'):  # Hidden variable
            assignments_list.append(range(self.states_count))
        else:  # Observation variable (fixed)
            t = int(var.split('_')[1])
            assignments_list.append([self.observation_sequence[t - 1]])

    all_assignments = list(itertools.product(*assignments_list))

    z_value = 0.0
    for assignment in all_assignments:
        assignment_dict = dict(zip(last_vars, assignment))
        z_value += last_clique['potential_func'](assignment_dict)

    return z_value
```

Key Steps:

1. **Clique Copying**: Create working copies to avoid modifying original potentials
2. **Forward Message Passing**: Iterate through the junction tree chain, computing and in-

corporating messages

3. **Message Computation**: For each edge, compute the message by summing over variables not in the separation set
4. **Potential Update**: Dynamically modify each clique's potential function to include incoming messages
5. **Final Summation**: After all messages are incorporated, sum over the final clique's potential

The algorithm ensures that after processing all messages, the final clique's potential represents the joint probability of all variables, and summing over it gives the partition function Z.

1.3.2 compute_marginals Implementation

The marginal probabilities computation uses forward-backward message passing:

```
def compute_marginals(self):
    # Step 1: Forward pass (left to right) - incorporate messages as we go
    forward_messages = {}
    working_cliques_forward = [clique.copy() for clique in self.cliques]

    for i in range(len(working_cliques_forward) - 1):
        from_clique = working_cliques_forward[i]
        to_clique = working_cliques_forward[i + 1]

        # Compute and store forward message
        message = self.compute_message(from_clique, to_clique)
        forward_messages[(from_clique['name'], to_clique['name'])] = message

        # Incorporate message into receiving clique's potential
        sepset_vars = self.get_sepset_variables(from_clique, to_clique)
        old_potential_func = to_clique['potential_func']

        def new_potential_func(assignments, old_func=old_potential_func,
                               msg=message, sep_vars=sepset_vars):
            sep_assignment = tuple(assignments[var] for var in sep_vars)
            message_value = msg.get(sep_assignment, 0.0)
            return old_func(assignments) * message_value

        to_clique['potential_func'] = new_potential_func

    # Step 2: Backward pass (right to left) - incorporate messages as we go
    backward_messages = {}
    working_cliques_backward = [clique.copy() for clique in self.cliques]

    for i in range(len(working_cliques_backward) - 1, 0, -1):
        from_clique = working_cliques_backward[i]
        to_clique = working_cliques_backward[i - 1]

        # Compute and store backward message
        message = self.compute_message(from_clique, to_clique)
        backward_messages[(from_clique['name'], to_clique['name'])] = message

        # Incorporate message into receiving clique's potential
```

```
        sepset_vars = self.get_sepset_variables(from_clique, to_clique)
        old_potential_func = to_clique['potential_func']

        def new_potential_func(assignments, old_func=old_potential_func,
                               msg=message, sep_vars=sepset_vars):
            sep_assignment = tuple(assignments[var] for var in sep_vars)
            message_value = msg.get(sep_assignment, 0.0)
            return old_func(assignments) * message_value

        to_clique['potential_func'] = new_potential_func

    # Step 3: Compute beliefs for each clique
    clique_beliefs = {}
    for clique in self.cliques:
        belief = {}
        clique_vars = clique['variables']

        # Generate all possible assignments for this clique
        assignments_list = []
        for var in clique_vars:
            if var.startswith('H_'):  # Hidden variable
                assignments_list.append(range(self.states_count))
            else:  # Observation variable (fixed)
                t = int(var.split('_')[1])
                assignments_list.append([self.observation_sequence[t - 1]])

        all_assignments = list(itertools.product(*assignments_list))

        for assignment in all_assignments:
            assignment_dict = dict(zip(clique_vars, assignment))

            # Start with clique's original potential
            belief_value = clique['potential_func'](assignment_dict)

            # Multiply with unconsumed forward messages
            clique_index = self.cliques.index(clique)
            for i in range(clique_index):
                from_clique = self.cliques[i]
                message_key = (from_clique['name'], clique['name'])
                if message_key in forward_messages:
                    sepset_vars = self.get_sepset_variables(from_clique, clique)
                    sep_assignment = tuple(assignment_dict[var] for var in sepset_vars)
                    belief_value *= forward_messages[message_key].get(sep_assignment, 0.0)

            # Multiply with unconsumed backward messages
            for i in range(clique_index + 1, len(self.cliques)):
                from_clique = self.cliques[i]
                message_key = (from_clique['name'], clique['name'])
                if message_key in backward_messages:
                    sepset_vars = self.get_sepset_variables(from_clique, clique)
                    sep_assignment = tuple(assignment_dict[var] for var in sepset_vars)
                    belief_value *= backward_messages[message_key].get(sep_assignment, 0.0)
```

```
            belief[tuple(assignment)] = belief_value

        clique_beliefs[clique['name']] = belief

    # Step 4: Compute marginals for each hidden variable
    marginals = []
    for t in range(1, self.num_observations + 1):
        for m in range(1, self.factors_count + 1):
            var_name = f'H_{t}^{m}'
            marginal = [0.0] * self.states_count

            # Find cliques containing this variable
            containing_cliques = [clique for clique in self.cliques
                                  if var_name in clique['variables']]

            if containing_cliques:
                clique = containing_cliques[0]
                belief = clique_beliefs[clique['name']]

                # Marginalize over other variables
                for assignment, belief_value in belief.items():
                    assignment_dict = dict(zip(clique['variables'], assignment))
                    if var_name in assignment_dict:
                        k_value = assignment_dict[var_name]
                        marginal[k_value] += belief_value

                # Normalize
                total = sum(marginal)
                if total > 0:
                    marginal = [p / total for p in marginal]
                else:
                    marginal = [1.0 / self.states_count] * self.states_count
            else:
                marginal = [1.0 / self.states_count] * self.states_count

            marginals.append(marginal)

    return marginals
```

Key Steps:

1. **Forward Pass**: Process left to right, storing messages and incorporating them into clique potentials
2. **Backward Pass**: Process right to left, doing the same in reverse direction
3. **Belief Computation**: For each clique, combine original potential with unconsumed messages
4. **Marginal Extraction**: For each variable, find a containing clique and marginalize its belief
5. **Normalization**: Ensure proper probability distributions

The implementation carefully handles the message passing to ensure that after both forward and backward passes, each clique's belief represents the correct joint distribution, from which marginals can be extracted by summation.

1.3.3 compute_top_k Implementation

The top-k assignments are computed using a message-passing procedure inspired by the max-product algorithm, extended to maintain multiple candidate paths. Instead of summing over hidden states (as in the partition function computation), the algorithm propagates and combines the k most probable partial assignments forward through the junction tree.

```
def compute_top_k(self, k):
    # Each entry: (log_prob, assignments_dict)
    top_k_paths = [(0.0, {})]

    for clique in self.cliques:
        new_paths = []
        vars_in_clique = clique['variables']

        # Build assignment domains
        domains = []
        for var in vars_in_clique:
            if var.startswith('H_'):
                domains.append(range(self.states_count))
            else:  # Observation variable
                t = int(var.split('_')[1])
                domains.append([self.observation_sequence[t - 1]])

        all_assignments = list(itertools.product(*domains))

        # Extend previous paths with new assignments
        for log_prob, path in top_k_paths:
            for assignment in all_assignments:
                assignment_dict = dict(zip(vars_in_clique, assignment))
                merged = path.copy()
                merged.update(assignment_dict)
                prob = clique['potential_func'](merged)
                if prob > 0:
                    new_paths.append((log_prob + math.log(prob), merged))

        # Keep only top-k paths
        top_k_paths = heapq.nlargest(k, new_paths, key=lambda x: x[0])

    # Convert back from log-space
    return [(math.exp(log_prob), path) for log_prob, path in top_k_paths]
```

Key Steps:

1. **Initialization**: Start with an empty assignment and probability 1 (log-probability 0).
2. **Clique Iteration**: Process each clique sequentially along the junction tree.
3. **Assignment Expansion**: For each clique, enumerate all possible assignments of hidden and observed variables.
4. **Path Extension**: Combine current candidate paths with new clique assignments, updating log-probabilities.
5. **Top-k Selection**: Use a heap to efficiently retain only the k most probable paths after each step.

6. **Final Conversion**: Convert the log-probabilities back to standard probabilities for output.

This algorithm ensures that at the end of message passing, we obtain the k most probable full assignments of hidden variables across all time steps, each consistent with the observed sequence. The use of log-probabilities prevents numerical underflow and the heap structure guarantees efficient selection of the top-k trajectories.

2 Loopy Belief Propagation

In the course, we have learnt various algorithms to answer inference queries. We started from algorithms like variable elimination, brute force inference, and then went on to the family of message passing algorithms, including the belief propagation algorithm and junction trees. In FHMMs, loops naturally arise due to the combination of multiple hidden chains and shared emission factors, so exact BP cannot be used. As mentioned in the problem statement, to solve inference problems in loopy graphs, we use the junction tree algorithm, which provides the exact inference, but its complexity becomes exponential.

2.1 Background

Loopy belief propagation applies the same message-passing rules as in trees, but on graphs with cycles. Since messages may circulate indefinitely around loops, the algorithm is typically run iteratively until the messages converge, that is, the update in message in a given iteration falls below a certain threshold, or until a maximum number of iterations is reached.

$$b_a(x_a) \propto f_a(X_a) \prod_{i \in Neighbour(a)} m_{i \to a}(x_i)$$

$$b_i(x_i) \propto f_i(x_i) \prod_{a \in Neighbour(i)} m_{a \to i}(x_i)$$

$$m_{i \to a}(x_i) = \prod_{c \in Neighbour(i) \setminus a} m_{c \to i}(x_i)$$

$$m_{a \to i}(x_i) = \sum_{X_a \setminus x_i} f_a(X_a) \prod_{j \in Neighbour(a) \setminus i} m_{j \to a}(x_j)$$

Figure 1: Message Passing in LBP

The messages are just beliefs of every node over a specific state, and the "updates" are based on the messages coming from neighbouring nodes (the equations governing this update are depicted in the above figure taken from [3]). After some iterations the each node computes its belief from the incoming message. So, LBP is an approximate inference algorithm.

2.2 Convergence & Complexity

Due to loops causing repeated influence and oscillating updates, the LBP algorithm might not always converge and there is a good chance that when it does, it converges to a local fixed point rather than the true marginals. However, it has been found to perform well enough in practice and as we will see from the next question of the assignment, it has found new interest in the academic community due to its applications in error correcting codes and other areas like computer vision and natural language processing.

As for the complexity of the algorithm, assuming $\mathcal{O}(1)$ number of iterations to convergence, we analyse the complexity as shown below: Let the maximum factor degree be D_f

Now, for a single message update, the cost is proportional to $\mathcal{O}(k^{D_f})$, since each factor has to sum over all possible assignments of its neighbouring variables (except one). In chain-structured models like FHMMs, most factors will be pairwise do $D_f = 2$, implying that each update will

cost $\mathcal{O}(\|^{\in})$. Each variable has a constant number of factors. Assuming the maximum number of factors the variables have to be m, the complexity should turn out to be $\mathcal{O}(m.k^{D_f})$

Now, considering FHMMs, since the with pairwise transitions and emission factors of degree F, this becomes:

- Transition Factors: $\mathcal{O}(TF.k^2)$
- Emission Factors: $\mathcal{O}(Tk^F)$

which makes the overall complexity $\mathcal{O}(T(F.k^2 + k^F))$ which is a huge improvement over the Junction Tree algorithm which has complexity $\mathcal{O}(k^{FT})$.

2.3 Implementation

The algorithm was implemented in Python using only standard libraries that were specified in the problem statement. Any use of LLMs has been specified in the code itself. The implementation was structured into three subroutines:

1. Factor to variable message updates
2. Variable to factor message updates
3. Marginal computation from converged messages

The messages were stored in *log-space* and normalised at each step to prevent underflow errors. *Damping* was also used, where updated messages were blended with previous messages according to

$$m^{(t)} = (1 - \lambda)\, m^{(t-1)} + \lambda\, m^{\text{new}},$$

with damping parameter $\lambda \in (0, 1]$. This gave a slightly improved convergence behaviour. On comparing these results with those obtained from exact inference on small instances, where exact computation is feasible, the marginals matched closely in practice, though small discrepancies appeared (errors of the order of 10^{-4}), which was expected as LBP is an approximate inference algorithm.

3 Turbo Codes

3.1 Background

Turbo Codes are some complicated encoding technique. They use two RSC - Recursive Systematic Convolutional encoders. We read up about encoders, convolutional encoders, systematic encoders, then finally recursive systematic convolutional encoders. Turbo codes essentially consist of the original systematic bitstring, one parity codeword, and one parity codeword generated by a permuted version of the bitstring, all 3 interleaved with each other. The shuffling and interleaving basically help in security against long burst errors. The code is sent through a noisy channel, which in this question is simulated by a probability matrix such that

$$probability_matrix[bit][v] = Pr[recvd_voltage = v | encoded_bit = bit]$$

This is a simulation of a discrete version of the errors so we have assumed that the noise is such that the received noisy output only consists of certain discrete voltage levels.

3.2 Trellis Generation

In encoding, a trellis refers to a graphical representation of a convolutional encoder or a trellis-coded modulation (TCM) scheme. It shows the encoder's possible states and the transitions between them over time, with each path representing a sequence of encoded symbols. Trellises provide a way to visualize the entire encoding process, allowing for efficient decoding algorithms like the Viterbi algorithm. The following is the code used for trellis generation, given the two generators being used for the turbo code encoding:

```
def generate_trellis(generator1, generator2, history_length=3):
    # Convert generators into binary masks (reversed for easy zip alignment)
    feedforward = bin(generator1)[2:][::-1].zfill(history_length)
    feedback = bin(generator2)[2:][::-1].zfill(history_length)

    num_states = 2 ** (history_length - 1)
    trellis = {}

    for state in range(num_states):
        # Represent state as deque of bits (length history_length-1)
        state_bits = [int(x) for x in format(state, f"0{history_length-1}b")]
        trellis[state] = {}

        for input_bit in [0, 1]:
            # History = [oldest, ..., newest]
            history = deque(state_bits + [input_bit], maxlen=history_length)

            # Feedback calculation
            feedback_val = sum(int(b) * int(g) for b, g in zip(history, feedback)) % 2

            # Replace last (input) with feedback result
            history[-1] = feedback_val

            # Parity calculation
            parity = sum(int(b) * int(g) for b, g in zip(history, feedforward)) % 2

            # Next state = drop oldest, keep last (history_length-1) bits
```

```
            next_state_bits = list(history)[1:]
            next_state = int("".join(str(x) for x in next_state_bits), 2)

            # Output = (systematic = input_bit, parity)
            trellis[state][input_bit] = (next_state, (input_bit, parity))

    return trellis
```

3.3 Viterbi Algorithm

The Viterbi algorithm is a classic algorithm used in various graphical model situations. As mentioned in the reference slides, we have used the trellis to find the most likely sequence of bits from the given noisy output. There were two mentioned decoding, one using hard-decisions, one using soft-decisions. The hard-decision decoding loses out on some information due to rounding off the voltage levels to bits 0 or 1 directly regardless of the voltage value. The soft-decision decoding takes into account the value of the voltage and computes the maximum likelihood based on that information. An important point to note is that we are only using the first parity code here because we are just trying to decode using the RSC bit encoded message using Viterbi algorithm, so we have discarded the parity2 string which is interleaved in the encoded string.
The two metrics that are computed are namely:

- Branch metric (BM) : captures tha value of negative log likelihood, i.e. negative log probability that we receive rcvd, given that xmit was sent.
- Path metric (PM) : PM[s,i] for each state s of the 2K-1 transmitter states and bit time i , where $0 \leqslant i < L = \text{len(message)}$.
  – PM[s,i] = smallest sum of BM(xmit, rcvd) over all message sequences m that place transmitter in state s at time i
  – PM[s,i+1] computed from PM[s,i] and p0[i],. . . ,pr-1[i]

Once these lists are filled, the algorithm backtracks the path with the last PM at the last level of the trellis graph to find the most probable sequence of bits taken. The decoded bits are then returned! Yay!
Time Complexity:
Clearly visible from the forward propogation loop of this algorithm, the time complexity is **O(n * S)** where n is the length of the encoded bitstring (systematic bits) and S is the number of states in the generated trellis i.e. $2^{history_length}$. Thus, it is O(n * $2^{history_length}$).
We have not exactly made any original optimizations but just used soft-decisions instead of hard-decisions mentioned in the reference slides thus giving more accurate results.

3.4 Turbo Codes and BCJL Algorithm

While the Viterbi algorithm focuses on calculating the most probable sequence of bits given the noisy output, the BCJL algorithm, for some k^{th} position, computes the probability that the bit is 0 or 1 given the entire received noisy output. Thus, this is more informative than the sequence of bits. The BCJL algorithm is also used first only on one parity code. It gives marginally better results than Viterbi.
The `get_turbocode_output(self)` funciton then uses two of such decoders with taking feedback from each other to converge to the perfectly decoded turbo code.

The BCJR algorithm computes the *a posteriori probability* (APP) of each information bit u_t given the received sequence y:

$$P(u_t = b \mid y), \quad b \in \{0, 1\}.$$

In practice, this is expressed in terms of the log-likelihood ratio (LLR):

$$L(u_t) = \ln \frac{P(u_t = 1 \mid y)}{P(u_t = 0 \mid y)}.$$

The computation relies on three metrics defined on the encoder trellis:

- Forward metric $\alpha_t(s)$: probability of being in state s at time t given all past observations.
- Backward metric $\beta_t(s)$: probability of observing the future sequence starting from state s at time t.
- Branch metric $\gamma_t(s, s')$: probability of a transition from s to s' given the current observation.

The APP is then obtained by summing over all transitions corresponding to the bit value:

$$P(u_t = b \mid y) \propto \sum_{(s,s'):u_t=b} \alpha_t(s)\, \gamma_t(s, s')\, \beta_{t+1}(s').$$

Turbo decoding employs two such BCJR decoders working iteratively. Each decoder uses the received systematic bits, one set of parity bits, and a priori information provided by the other decoder. After each BCJR pass, *extrinsic information* is extracted:

$$L^{\text{ext}}(u_t) = L(u_t) - L^{\text{ch}}(u_t) - L^{\text{a-priori}}(u_t),$$

which is interleaved or de-interleaved and passed to the companion decoder. Point to be noted: for the decoder dealing with the shuffled parity code, the extrinsic information and systematic code is also shuffled to match the corresponding encoder. Then the extrinsic information from that decoder is unshuffled to pass on to the first decoder and so on. Over successive iterations, the APPs are refined until convergence.

Stopping criteria include a fixed maximum number of iterations which for our case is `epochs = 7`. The time complexity of BCJR algorithm is the same as Viterbi i.e. O(n * S) and the space complexity is O(n * S) too. For turbo decoding, the same is done for `epoch * 2` times.

3.5 Comparing Performance

As visible from the results, Viterbi and BCJR algorithm both have quite similar bit error rates. BCJR is marginally better than Viterbi, though. For the provided test cases, using Viterbi and BCJR algorithms, there are upto around 10% errors in the decoded strings.

The Turbo code output is the best, as one would expect, using two decoders and feedback from each other using the two parity codes interleaved with the systematic bits. For the given test cases, Turbo decoder works perfectly giving no errors in all given cases.

Considering a hybrid algorithm, which uses the a posteriori probabilities generated by the BCJR algorithm fed to Viterbi algorithm, this will probably not be much better than the regular turbo decoding since in large blocklengths, interleaving + iterative decoding "averages out" noise. Although, in short blocks, a couple of unlucky noise spikes can dominate greedy decisions. Thus, the hybrid algorithm might be better for short block lengths. In practice, this algorithm should be worse since it does not give much edge over the original algorithm and it increases the time and space complexity of the decoding algorithm.

References

[1] Zoubin Ghahramani and Michael I. Jordan. Factorial hidden markov models. Technical Report AIM-1561, MIT Artificial Intelligence Laboratory, 1997.

[2] Alexander T. Ihler, John W. Fisher III, and Alan S. Willsky. Loopy belief propagation: Convergence and effects of message errors. *J. Mach. Learn. Res.*, 6:905–936, 2005.

[3] Eric P. Xing, Rajarshi Das, Zhengzhong Liu, and Dishan Gupta. 13: Variational inference: Loopy belief propagation, 2014. Lecture notes for Carnegie Mellon University course 10-708: Probabilistic Graphical Models, Spring 2014.